To do this, I have a copy of the Lubuntu file system copied in the path specified below. The same can be seen in the screenshot provided in Figure 1, as I have *touched* HOST_FS and CONTAINER_FS as two files within the root of the host and within the copy of our Lubuntu FS.

We will now have to let our container know about this file system and ask it to change its root to this copied file system. We will also have to ask the container to change its directory to / once it's launched.

```
func child() {
    // Stuff that we previously went
over

    must(syscall.Chroot("/home/lubuntu/
Projects/make-sense-of-containers/
lubuntu-fs"))
    must(syscall.Chdir("/"))
    must(cmd.Run())
}
```

Upon running this, we get our intended file system. We can confirm it in Figure 2 as we can see CONTAINER_FS, the file that we created in our container.

However, despite all of these efforts, *ps* still remains a problem.

This is because while we provided a new file system for our container using *chroot*, we forgot that */proc* in itself is a special type of virtual file system.

/proc is sometimes referred to as a process information pseudo-file system. It doesn't contain 'real' files but runtime system information like system memory, devices mounted, hardware configuration, etc. For this reason, it can be regarded as a control and information centre for the kernel. In fact, quite a lot of system utilities are simply calls to files in this directory. For example, *lsmod* is the same as *cat /proc/modules*. By altering files located in this directory you can even read/change kernel parameters like *sysctl* while the system is still running.

```
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# ps
    PID TTY          TIME CMD
  23550 pts/0    00:00:00 sudo
  23551 pts/0    00:00:00 bash
  23560 pts/0    00:00:00 ps
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# ls /
bin   dev  home     lib    lib64   lost+found  mnt  proc  run   snap  sys  usr
boot  etc  HOST_FS  lib32  libx32  media       opt  root  sbin  srv   tmp  var
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# go run main.go run bash
Running [bash] as PID 1
root@lubuntu-vm:/# ls /
bin   CONTAINER_FS  etc   lib    lib64   media  opt   root  sbin  srv  tmp  var
boot  dev           home  lib32  libx32  mnt    proc  run   snap  sys  usr
root@lubuntu-vm:/# ps
Error, do this: mount -t proc proc /proc
root@lubuntu-vm:/#
```

Figure 2: Confirming that our container is now using its own file system. However, `ps` is still not working

```
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# mount | grep proc
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=28,pgrp=1,timeout=0,minproto=5,ma
xproto=5,direct,pipe_ino=1480)
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# go run main.go run bash
Running [bash] as PID 1
root@lubuntu-vm:/# mount
proc on /proc type proc (rw,relatime)
root@lubuntu-vm:/#

root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers# mount | grep proc
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
systemd-1 on /proc/sys/fs/binfmt_misc type autofs (rw,relatime,fd=28,pgrp=1,timeout=0,minproto=5,ma
xproto=5,direct,pipe_ino=1480)
proc on /home/lubuntu/Projects/make-sense-of-containers/lubuntu-fs/proc type proc (rw,relatime)
root@lubuntu-vm:/home/lubuntu/Projects/make-sense-of-containers#
```

Figure 3: Observing output of *`mount | grp` proc* from the host before and after running our container

Hence, we need to mount */proc* for our *ps* command to be able to work.

```
func child() {
    // Stuff that we previously went
over
                must(syscall.Chroot("/
home/lubuntu/Projects/make-sense-of-
containers/lubuntu-fs"))
    must(syscall.Chdir("/"))
    // Parameters to this syscall.
Mount() are:
    // source FS, target FS, type of
the FS, flags and data to be written in
the FS
    must(syscall.Mount("proc", "proc",
"proc", 0, ""))

    must(cmd.Run())

    // Very important to unmount in the
end before exiting
    must(syscall.Unmount("/proc", 0))
}
```

You can think of *syscall.Mount()* and *syscall.Unmount()* as the functions that are called when you plug-in and safely remove a pen-drive. In the same way, we *mount* and *unmount* our */proc* file system in our container.

After all these efforts, now if we run *ps* from our container, we can finally see that we have PID 1! This means that we have finally achieved process isolation. We can see that our */proc* file system has been mounted by doing *ls /proc*, which lists the current process information of our container.

One small thing that we need to check is to see the mount points of *proc*. We will do that by first running *mount | grep proc* from our host OS. We will then launch our container and run the same command again. With our container still running, we will once again run *mount | grep proc to* check the mount points of *proc*.

As we can see in Figure 3, if we run *mount | grep proc* from our host operating system (OS) with our container running, the host OS can see where *proc* is mounted in our container. This should not be the case. Ideally, our containers should be as transparent to the host OS as possible. To fix this,